

LITTLE SNITCH ALERT TRIAGE

How to decide: genuine system connection, legitimate app, or compromise?

CONTENTS

1. Two-second triage rule
2. The alert window – what every field tells you
3. The 5-question decision tree
4. Signature verification (the only thing that doesn't lie)
5. File-path heuristics (where legit binaries live vs. where malware hides)
6. Domain & remote-host heuristics
7. Known-legitimate Apple processes (don't panic about these)
8. Strong red flags – likely compromise
9. Spoofed alerts and UI phishing
10. Verification commands (copy-paste reference)
11. Decision rules for the "Allow / Deny" buttons
12. When in doubt: a deeper investigation workflow

1. TWO-SECOND TRIAGE RULE

If you can't answer ALL of the following in two seconds, default to "Deny – Until Quit" and investigate before allowing "Forever":

- (a) Do I recognize the process making the connection?
- (b) Does the process path look right for that process?
- (c) Is the remote host plausibly owned by the people who make this process?

If any answer is "no" or "I don't know," DENY for now. You can always re-allow later. You CAN'T un-exfiltrate data.

2. THE ALERT WINDOW – WHAT EVERY FIELD TELLS YOU

A genuine Little Snitch alert shows:

Process	: the binary name (e.g., "curl", "Safari", "WindowServer")
Code requirement:	how the binary is signed
	- "anchor apple" = Apple-signed (system)
	- "anchor apple generic and..." = Developer-ID signed third party
	- "designated => identifier ..." = ad-hoc / self-signed (suspicious)
	- (none / "no code signature") = unsigned (strong red flag)
Server	: either a hostname or an IP address
Port	: 443/80 is normal HTTPS/HTTP; high ports need a reason
Process path	: full path on disk – read it carefully (see §5)
Parent process	: who started this binary – often the clue to spoofing
Notes	: descriptions LS pulled from its rule library

Click the disclosure triangle / info button. If the alert HIDES details or the info button does nothing, that's a red flag – see §9.

3. THE 5-QUESTION DECISION TREE

Walk these in order. Stop at the first DENY.

- Q1. Is the code signature valid AND from the expected developer?
- "anchor apple" for an Apple-named process? pass
 - "anchor apple generic" + matching Team ID for a third party (e.g. Mozilla = 43AQ936H96)? pass
 - Ad-hoc / unsigned / "no signature"? DENY
- Q2. Does the binary live in a sensible path? (see §5)
- /System/, /usr/libexec/, /usr/bin/, /usr/sbin/, /Applications/<Foo>.app/Contents/MacOS/<Foo> pass
 - /tmp, /private/tmp, /var/folders/.../T/, /Users/Shared, ~/.cache, ~/.local, hidden dotfiles ... DENY
- Q3. Does the process name match a known-good binary, or is it imitating one (Safari, WindowServer, etc.)?
- Exact known name (Safari, WindowServer, kernel_task) . pass
 - Off-by-one or appended letter DENY
- Q4. Is the remote host plausible for this process?
- apsd → push.apple.com, courier.push.apple.com? pass
 - Safari → arbitrary HTTPS hosts on 443? pass (browser)
 - "curl" or unknown binary → bare IP in unknown country? DENY
- Q5. Have I seen this same alert several times today and already approved it? OR is this the first time?
- First time + unknown app = pause; do NOT "Allow Forever"
 - First time = "Allow Until Quit" at most until verified

4. SIGNATURE VERIFICATION – THE ONLY THING THAT DOESN'T LIE

Run these in Terminal against the path shown in the LS alert:

```
codesign -dv --verbose=4 "/path/from/alert"
spctl -a -vv "/path/from/alert"
codesign --verify --deep --strict --verbose=2 "/path/from/alert"
```

What to look for:

Authority=Apple Root CA	-> Apple OS binary
Authority=Developer ID Application: ...	-> third-party, signed
TeamIdentifier=<10 hex chars>	-> developer's Team ID
spctl: accepted	-> Gatekeeper trusts it
spctl: rejected, source=Unnotarized	-> not notarized (suspicious for new app)
errSecCSUnsigned / no signing	-> UNSIGNED – strong red flag

Cross-check the Team ID against the developer's known ID. Apple's Team ID is literally "APPLE." Major vendors:

Mozilla Firefox	43AQ936H96
Google Chrome	EQHXZ8M8AV
Microsoft	UBF8T346G9
1Password	2BUA8C4S2C
Signal	U68MSDN6DR
Tor Browser	MADPSAYN6T

If the alert shows "Firefox" but TeamIdentifier is anything other than 43AQ936H96, IT IS NOT FIREFOX.

5. FILE-PATH HEURISTICS

Legitimate locations (green):

/System/Library/	Apple system frameworks/daemons
/System/Applications/	Apple bundled apps
/usr/libexec/	Apple helper binaries
/usr/sbin/ /usr/bin/	Apple system tools
/Applications/	installed apps (check signature!)
/Library/Apple/	Apple-managed
~/Applications/	per-user installs (check signature!)
/opt/homebrew/ /usr/local/	Homebrew (check brew owns it)
/Applications/<X>.app/Contents/MacOS/	inside-bundle executables

Suspicious locations (yellow → investigate):

/Users/Shared/	rare for real apps
~/Library/LaunchAgents/<weird name>	persistence directory – check signature
~/Library/LaunchDaemons/	ditto
~/Library/Application Support/<X>/	only OK if the parent app is real

Hostile locations (red → almost certainly malicious):

/tmp/ /private/tmp/	temp dirs – no legit persistent binary
/var/folders/.../T/	user temp dirs
/private/var/folders/	same
~/ .cache / ~/ .local/	Linux conventions – uncommon on macOS
Anywhere with a UUID-like name	e.g., /tmp/3F2A8B-9C4D-...
Names with extra dots or trailing spaces: "Safari ", "Safari.", ".Safari"	

6. DOMAIN & REMOTE-HOST HEURISTICS

GREEN (typical legitimate destinations):

- *.apple.com / icloud.com / mzstatic.com / cdn-apple.com
- *.cloudfront.net (Amazon CDN – many legit apps use this)
- *.akamaihd.net / *.akamaized.net (Akamai CDN)
- *.fastly.net (Fastly CDN)
- *.googleusercontent.com (Google)
- *.microsoft.com / live.com / office.com
- github.com / objects.githubusercontent.com (GitHub)
- formulae.brew.sh / ghcr.io (Homebrew)

YELLOW (legitimate context-dependent – verify it matches the process):

- Bare IPv4/IPv6 addresses
- *.ngrok.io / *.dyndns.org / dynamic DNS providers
- Country TLDs unrelated to the app (.ru, .cn, .top, .xyz)
- Domains < 30 days old (use `whois <domain>`)
- *.onion (Tor) – only OK if it's Tor Browser making the request

RED (assume hostile until proven otherwise):

- Typo-squat of a known brand (appie.com, apple-cloudkit.io, icloud-apple.net)
- Punycode/IDN homoglyph: xn--... in raw URL form
- Hex-encoded subdomain or DGA-style: dn4q72k3.example.com
- Pure-IP destination on a high port (>1024) to an unknown country

Telegram bot endpoint (api.telegram.org) called by anything that's not Telegram

7. KNOWN-LEGITIMATE APPLE PROCESSES (don't panic)

These are normal and should be allowed for their normal destinations:

apsd	Apple Push: push.apple.com, courier.push.apple.com
configd	DHCP, network config
trustd	cert revocation (ocsp.apple.com, valid.apple.com)
mDNSResponder	local Bonjour + DNS for everything
symptomsd	network reachability telemetry
identityservicesd	iMessage, FaceTime, iCloud
imagent / IMTransfer	iMessage
nsurlsessiond	background URL sessions for apps
CommCenter	cellular (only on iPad/iPhone-class device)
cloudd / bird	CloudKit / iCloud Drive
storeagent / storeassetd	App Store / Music store
akd	Apple ID authentication
geod	Apple location services
parsecd	Siri suggestions
syspolicyd	Gatekeeper / notarization checks
helpd	Help Viewer content
apsctl	Apple Push helper
WindowServer	display server (rarely opens net but can for AirPlay)
softwareupdated	system software updates
mobileassetd	asset/dictionary/font downloads
ManagedClient	MDM (only if your device is managed)
ContentCachingService	if Content Caching is on
rapportd	Continuity / AirDrop coordination
searchpartyd	Find My
amsengagementd	App Store engagement
swcd	associated domains (universal links)
netbiosd	SMB browsing (we disabled this in lock-remote-access)

8. STRONG RED FLAGS – TREAT AS COMPROMISE UNTIL PROVEN OTHERWISE

Any ONE of these is enough to deny "Forever" and start investigating:

- [] Binary path under /tmp, /private/tmp, or /var/folders/.../T/
- [] Process name imitates a system process (off-by-one letter, extra "d")
- [] Code signature is "ad-hoc" or "(unsigned)"
- [] TeamIdentifier doesn't match the named vendor
- [] Destination is a bare IP in a country unrelated to the service
- [] Periodic beaconing – alerts at exactly N-minute intervals
- [] First connection happens immediately after waking from sleep, to an unknown host, before you've launched anything
- [] Parent process is a shell (bash/zsh/sh) but the named process is a GUI app
- [] LS alert appears with NO entry in the LS Network Monitor's flow log (the alert may be spoofed – see §9)
- [] Connecting to Telegram / Discord / Pastebin / paste-style sites from a process that has no business there
- [] DNS queries containing base64-like or hex-like long random strings (DGA / data exfiltration via DNS)

9. SPOOFED ALERTS AND UI PHISHING

Real malware CAN draw a window that looks like a Little Snitch alert to trick you into typing your admin password or "approving" something. Defenses:

- Real LS alerts are produced by the LS network filter and have a matching entry in LS Network Monitor → "All Connections." Open Network Monitor: if the alert exists but there is NO corresponding flow, the alert is fake.
- Real LS alerts NEVER ask for your admin password. If a dialog that looks like LS asks for credentials, it is phishing.
- Real LS alerts do not appear without a triggering process. If you see a "LS" alert and Activity Monitor shows nothing matching the listed process, it's fake.
- Real LS dialogs respond to Cmd-Q on the Little Snitch Agent app (they're produced by it). A spoofed alert often won't be tied to that process group.
- When in doubt: hit "Deny," then open Network Monitor from the LS menu-bar item and verify the flow is real.

10. VERIFICATION COMMANDS (COPY-PASTE)

Replace <path> with the process path the alert shows.

```
# signature
codesign -dv --verbose=4 "<path>"
codesign --verify --deep --strict --verbose=2 "<path>"
spctl -a -vv "<path>"
```

```
# what the process is doing right now
pgrep -fl "<process name>"
PID=$(pgrep -f "<process name>" | head -1)
ps -o pid,ppid,user,etime,command -p $PID
sudo lsof -n -p $PID | grep -E "TCP|UDP"
```

```
# remote host details - careful, this queries DNS/whois
dig +short <host>
whois <domain> | head -30
```

```
# hash of the binary (compare to known-good, e.g., from a clean Mac)
shasum -a 256 "<path>"
md5 "<path>"
```

```
# what brew owns it (if it's in /opt/homebrew or /usr/local)
brew list --verbose 2>/dev/null | grep "<basename of path>"
```

```
# what package owns it (Apple bundled stuff)
pkgutil --file-info "<path>"
```

```
# Gatekeeper origin
xattr -l "<path>" # look for com.apple.quarantine
```

```
# Little Snitch's own log
log show --predicate 'subsystem == "at.obdev.littlesnitch"' --last 1h | head -
```

11. DECISION RULES FOR THE "ALLOW / DENY" BUTTONS

Little Snitch offers several scopes when you allow or deny. Use them this way:

ALLOW - Until Quit	Default for "looks fine but I haven't verified"
ALLOW - For 1 hour	Useful when you're actively using something new
ALLOW - Forever	ONLY after you have: - verified the signature (codesign + spctl) - verified the path is sensible - confirmed the remote host belongs to the vendor
DENY - Until Quit	Default for anything you don't recognize
DENY - Forever	Use for any "shouldn't ever connect" choice (e.g., Adobe Reader updater, Office telemetry)

Rule of thumb: every "Allow Forever" you create is a future blind spot. Verify before you make a permanent rule.

12. WHEN IN DOUBT - DEEPER INVESTIGATION WORKFLOW

Step 1 - Deny the connection "Until Quit" (don't make it permanent).

Step 2 - In the Little Snitch alert, click the info button. Note:

- exact process path
- code requirement string
- parent process name
- exact destination host or IP

Step 3 - Open Terminal. Run:

```
codesign -dv --verbose=4 "<process path>"
spctl -a -vv "<process path>"
```

Decide: signed by whom?

Step 4 - Locate the binary in Finder:

```
open -R "<process path>"
```

Is the path normal? (See §5)

Step 5 - Identify the parent process:

```
ps -o ppid= -p $(pgrep -fn "<process>")
ps -o pid,command -p <that PPID>
```

A signed app launched by launchd or by another signed app is normal. A signed app launched by /bin/sh, curl, or a random binary in /tmp is NOT normal.

Step 6 - Check the destination:

```
whois <domain> | grep -iE "creat|regist|expir"
```

Newly-registered domains targeting "system" processes are a strong indicator of compromise.

Step 7 - If anything still feels wrong: power off Wi-Fi (don't reboot yet), capture the binary and a launchd plist if there is one, and contact a forensic responder. For nation-state level targeting:

- Access Now Digital Security Helpline (24/7, free)
<https://www.accessnow.org/help/>
- Citizen Lab (commercial-spyware focus)

<https://citizenlab.ca/>
- EFF
<https://www.eff.org/about/contact>

Step 8 – Do NOT upload suspicious binaries to VirusTotal or similar public sandboxes if you believe you are being targeted by a sophisticated adversary. The upload tells the attacker you are investigating and may burn their malware sample, prompting them to escalate. Share samples privately with a responder instead.

=====

QUICK CARD – KEEP THIS NEAR THE KEYBOARD

=====

GREEN Apple system path + "anchor apple" signature
Known third-party app + correct Team ID + sensible host

YELLOW Unknown app in /Applications/ with valid Developer ID
Bare IP destination – verify before allowing
Apple process talking to a non-Apple CDN

RED Binary in /tmp, /var/folders, /Users/Shared, hidden dotpath
Unsigned or ad-hoc signed binary
Process name imitating system process (off-by-one letter)
TeamIdentifier doesn't match named vendor
Bare IP to an unknown country / new domain
Telegram / Discord / pastebin destinations from system processes
"LS alert" with NO matching flow in Network Monitor

When in doubt: DENY Until Quit, investigate, then decide.
Never use Allow Forever for something you haven't verified.

=====

END

=====